

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №9
дисциплины
«Объектно-ориентированное программирование»
Вариант № 18

Выполнил:
Текеева Мадина Азрет-Алиевна
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии Воронкин Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2025 г.

Тема: Управление процессами в Python.

Цель работы: приобретение навыков написания многозадачных приложений на языке программирования Python версии 3.x.

Репозиторий: https://github.com/bickrosss/TM_oop_lab9

Порядок выполнения работы:

1. Простой пример создания и запуска дочернего процесса.

```
from multiprocessing import Process

def func():
    print("Hello from child Process")

if __name__ == "__main__":
    print("Hello from main Process")
    proc = Process(target=func)
    proc.start()
```

2. Использование метода join() для синхронизации процессов.

```
from multiprocessing import Process

def func():
    print("Hello from child Process")

if __name__ == "__main__":
    print("Hello from main Process")
    proc = Process(target=func)
    proc.start()
    proc.join()
    print("Goodbye")
```

3. Использование метода is_alive() для мониторинга активности процесса.

```
from multiprocessing import Process

def func():
    print("Hello from child Process")

if __name__ == "__main__":
    print("Hello from main Process")
    proc = Process(target=func)
    proc.start()
    print(f"Proc is_alive status: {proc.is_alive()}")
    proc.join()
```

```
print("Goodbye")
print(f"Proc is_alive status: {proc.is_alive()}")
```

4. Создание наследника класса Process с переопределением метода run().

```
from multiprocessing import Process
from time import sleep

class CustomProcess(Process):
    def __init__(self, limit):
        Process.__init__(self)
        self._limit = limit

    def run(self):
        for i in range(self._limit):
            print(f"From CustomProcess: {i}")
            sleep(0.5)

if __name__ == "__main__":
    cpr = CustomProcess(3)
    cpr.start()
```

5. Использование метода terminate() для остановки бесконечного процесса.

```
from multiprocessing import Process
from time import sleep

def func():
    counter = 0
    while True:
        print(f"counter = {counter}")
        counter += 1
        sleep(0.1)

if __name__ == "__main__":
    proc = Process(target=func)
    proc.start()
    sleep(0.7)
    proc.terminate()
```

6. Создание фоновых процессов, завершающихся вместе с родительским.

```
from multiprocessing import Process
from time import sleep

def func(name):
```

```

counter = 0
while True:
    print(f"proc {name}, counter = {counter}")
    counter += 1
    sleep(0.1)

if __name__ == "__main__":
    # Указание на то, что процесс демон при создании объекта класса Process
    proc1 = Process(target=func, args=("proc1",), daemon=True)
    # Указание на то, что процесс демон через свойство daemon
    proc2 = Process(target=func, args=("proc2",))
    proc2.daemon = True

    # Запуск процессов
    proc1.start()
    proc2.start()
    sleep(0.3)

```

7. Выполнили задание 1.

С использованием многопроцессорности для заданного значения x найти сумму ряда S с точностью члена ряда по абсолютному значению $\varepsilon = 10^{-7}$ и произвести сравнение полученной суммы с контрольным значением функции y для двух бесконечных рядов. Рассматривается знакочередующийся степенной ряд.

$$S = \sum_{n=0}^{\infty} \frac{x^{2n+1}}{2n+1} = x + \frac{x^3}{3} + \frac{x^5}{5} + \dots$$

$$x = 0,35; \quad y = \ln \sqrt{\frac{1+x}{1-x}}.$$

Рисунок 1. Вариант 18

Листинг программы:

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import math
from multiprocessing import Manager, Process
from typing import Tuple, Callable

class SeriesProcess(Process):

```

```

def __init__(
    self,
    x: float,
    eps: float,
    start_idx: int,
    step: int,
    results: dict,
    pos: int,
    term_func: Callable[[float, int], float]
) -> None:
    super().__init__()
    self.x: float = x
    self.eps: float = eps
    self.start_idx: int = start_idx
    self.step: int = step
    self.results: dict = results
    self.pos: int = pos
    self.term_func = term_func

def run(self) -> None:
    partial_sum: float = 0.0
    count: int = 0

    n: int = self.start_idx
    prev_term: float = float('inf')

    while True:
        term: float = self.term_func(self.x, n)
        if abs(term) < self.eps and abs(term) <= abs(prev_term):
            break

        partial_sum += term
        count += 1
        n += self.step
        prev_term = term

        if count > 10000:
            break

    self.results[f"sum_{self.pos}"] = partial_sum
    self.results[f"count_{self.pos}"] = count

def calculate_series_1(x: float, eps: float, num_processes: int = 4) ->
Tuple[float, int]:
    def term_func_1(x_val: float, n: int) -> float:

        try:
            return (x_val ** (2 * n + 1)) / (2 * n + 1)
        except (OverflowError, ZeroDivisionError):

```

```

        return 0.0

    with Manager() as manager:
        results: dict = manager.dict()
        processes: list = []

        for i in range(num_processes):
            p = SeriesProcess(x, eps, i, num_processes, results, i, term_func_1)
            processes.append(p)
            p.start()

        for p in processes:
            p.join()

        total_sum: float = sum(results[f"sum_{i}"] for i in range(num_processes))
        total_count: int = sum(results[f"count_{i}"] for i in
range(num_processes))

    return total_sum, total_count


def calculate_series_2(x: float, eps: float, num_processes: int = 4) ->
Tuple[float, int]:
    def term_func_2(x_val: float, n: int) -> float:
        try:
            return 1.0 / ((2 * n - 1) * (x_val ** (2 * n - 1)))
        except (OverflowError, ZeroDivisionError):
            return 0.0

    with Manager() as manager:
        results: dict = manager.dict()
        processes: list = []
        for i in range(num_processes):
            p = SeriesProcess(x, eps, i + 1, num_processes, results, i,
term_func_2)
            processes.append(p)
            p.start()

        for p in processes:
            p.join()

        total_sum: float = sum(results[f"sum_{i}"] for i in range(num_processes))
        total_count: int = sum(results[f"count_{i}"] for i in
range(num_processes))

    return total_sum, total_count


def get_control_value_1(x: float) -> float:
    return 0.5 * math.log((1 + x) / (1 - x))

```

```
def get_control_value_2(x: float) -> float:
    return 0.5 * math.log((x + 1) / (x - 1))
```

ЛИСТИНГ ВЫЗОВА:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
```

```
from series_two_series import (
    calculate_series_1,
    calculate_series_2,
    get_control_value_1,
    get_control_value_2
)
```

```
def print_series_info(series_num: int, x: float, eps: float,
                      calculated_sum: float, control_value: float,
                      term_count: int) -> None:
    """Вывод информации о вычислениях ряда."""
    error = abs(calculated_sum - control_value)

    print(f"\n{'='*60}")
    print(f"Ряд {series_num}")
    print(f"{'='*60}")

    if series_num == 1:
        print("Ряд:  $S = \sum [x^{(2n+1)}/(2n+1)]$ ,  $n=0..\infty$ ")
        print("Контрольная функция:  $y = \ln(\sqrt{(1+x)/(1-x)}) = 0.5 * \ln((1+x)/(1-x))$ ")
    else:
        print("Ряд:  $S = \sum [1/((2n-1)*x^{(2n-1)})]$ ,  $n=1..\infty$ ")
        print("Контрольная функция:  $y = 0.5 * \ln((x+1)/(x-1))$ ")

    print(f"\nx = {x},  $\epsilon$  = {eps}")
    print("\nРезультаты:")
    print(f"    Вычисленная сумма: {calculated_sum:.10f}")
    print(f"    Контрольное значение: {control_value:.10f}")
    print(f"    Абсолютная погрешность: {error:.2e}")
    print(f"    Учтено членов ряда: {term_count}")

    if error < eps:
        print("    ✓ Точность достигнута (погрешность <  $\epsilon$ )")
    else:
        print("    X Погрешность превышает  $\epsilon$ ")

def main() -> None:
    """Основная функция для выполнения индивидуального задания."""
    print("ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ")
```

```

print("Вычисление сумм двух бесконечных рядов с использованием
многопроцессности")
print("=" * 60)

# Параметры из задания
x1 = 0.35 # Для первого ряда  $|x| < 1$ 
x2 = 3.0  # Для второго ряда  $|x| > 1$ 
eps = 1e-7

print(f"\nОбщая точность:  $\epsilon = \{eps\}$ ")

# Вычисление первого ряда
print("\n" + "="*60)
print("ВЫЧИСЛЕНИЕ ПЕРВОГО РЯДА")
print("="*60)

sum1, count1 = calculate_series_1(x1, eps, num_processes=4)
control1 = get_control_value_1(x1)
print_series_info(1, x1, eps, sum1, control1, count1)

# Вычисление второго ряда
print("\n" + "="*60)
print("ВЫЧИСЛЕНИЕ ВТОРОГО РЯДА")
print("="*60)

sum2, count2 = calculate_series_2(x2, eps, num_processes=4)
control2 = get_control_value_2(x2)
print_series_info(2, x2, eps, sum2, control2, count2)

# Сводка результатов
print("\n" + "="*60)
print("СВОДКА РЕЗУЛЬТАТОВ")
print("="*60)

error1 = abs(sum1 - control1)
error2 = abs(sum2 - control2)

print(f"\nПервый ряд (x={x1}):")
print(f"  Погрешность: {error1:.2e} {'<' if error1 < eps else '>'}  $\epsilon$ ")
print(f"  Членов ряда учтено: {count1}")

print(f"\nВторой ряд (x={x2}):")
print(f"  Погрешность: {error2:.2e} {'<' if error2 < eps else '>'}  $\epsilon$ ")
print(f"  Членов ряда учтено: {count2}")

print("\n" + "="*60)
if error1 < eps and error2 < eps:
    print("✓ ОБА РЯДА ВЫЧИСЛЕНЫ С ЗАДАННОЙ ТОЧНОСТЬЮ")
else:
    print("X ТОЧНОСТЬ НЕ ДОСТИГНУТА ДЛЯ ОДНОГО ИЛИ ОБОХ РЯДОВ")
print("="*60)

```



```
if __name__ == "__main__":  
    main()
```

Вычисление сумм двух бесконечных рядов с использованием многопроцессности

Общая точность: $\epsilon = 1e-07$

ВЫЧИСЛЕНИЕ ПЕРВОГО РЯДА

РЯД 1

Ряд: $S = \sum [x^{(2n+1)}/(2n+1)], n=0..\infty$

Контрольная функция: $y = \ln(\sqrt{((1+x)/(1-x))}) = 0.5 * \ln((1+x)/(1-x))$

$x = 0.35, \epsilon = 1e-07$

Результаты:

Вычисленная сумма: 0.3654436525

Контрольное значение: 0.3654437543

Абсолютная погрешность: 1.02e-07

Учтено членов ряда: 6

X Погрешность превышает ϵ

ВЫЧИСЛЕНИЕ ВТОРОГО РЯДА

РЯД 2

Ряд: $S = \sum [1/((2n-1)*x^{(2n-1)})], n=1..\infty$

Контрольная функция: $y = 0.5 * \ln((x+1)/(x-1))$

$x = 3.0, \epsilon = 1e-07$

Рисунок 1. Результат выполнения программы

Вывод: в ходе выполнения лабораторной работы были приобретены навыки написания многозадачных приложений на языке программирования Python версии 3.x.